```
#
allow myapp_t myapp_log_t:file { read_file_perms append_file_perms };
#
allow myapp_t myapp_tmp_t:file manage_file_perms;
files_tmp_filetrans(myapp_t,myapp_tmp_t,file)
#
```

If we closely look at the above file, the valid statements for an selinux module file (as discussed in the previous articles) would be:

Type Declarations:

```
type myapp_t;
type myapp_exec_t;
type myapp_log_t;
type myapp_tmp_t;
```

and the allow rules:

```
allow myapp_t myapp_log_t:file { read_file_perms append_file_perms };
allow myapp_t myapp_tmp_t:file manage_file_perms;
```

But would this file compile? The syntax of the allow rule seems to be fine, but the permissions don't. Do we not know that the permissions are categorised as read, write, etc?

What kind of permissions are read_file_perms, append_file_perms and manage_file_perms?

Moreover, what are the other statements in this file, such as:

```
domain_type(myapp_t)
domain_entry_file(myapp_t, myapp_exec_t)
logging_log_file(myapp_log_t)
files_tmp_file(myapp_tmp_t)
files_tmp_filetrans(myapp_t,myapp_tmp_t,file)
```

Why are the above not terminated with a semi-colon? Will this not give a syntax error while compiling?

The answer to the above questions is 'macros'. Macros provide reusable code pieces and can be expanded to form valid syntax statements. Macros are written in the m4 macro language (the same one used for creating the sendmail configuration file - sendmail.cf). SELinux macros can be classified as capability macros and permission macros.

Capability macros are expanded to form allow rules, type transition rules, etc. Permission macros expand to valid permissions.

Looking at the above example.te file, the permission macros used in it are:

```
read_file_perms, append_file_perms and manage_file_perms
```

The capability macros used are:

```
domain_type, domain_entry_file, logging_log_file, files_tmp_file and files_tmp_filetrans.
```

But what do these macros do? How are they expanded? Can you only use these macros or define your own as well?

Macros are defined in *.*spt* files. These files are found by default in the folder /usr/share/selinux/devel/include/support, as can be seen below:

```
[vbg@vbg-work support]$ pwd
/usr/share/selinux/devel/include/support

[vbg@vbg-work support]$ ls
all_perms.spt  file_patterns.spt  loadable_module.spt  misc_patterns.spt
obj_perm_sets.spt  segenxml.py    segenxml.pyo
divert.m4      ipc_patterns.spt   misc_macros.spt      mls_mcs_macros.spt
policy.dtd         segenxml.pyc   undivert.m4
```

Macros can also be defined in the interface files that are included in a module, along with type enforcement and file contexts. The default extension for interface files is ".if" and many macros are defined here. For a better understanding of macros, let us expand the sample example.te file and create a type enforcement file without these macros, but with raw permissions and directives/statements in it.

Permission macros

Let us first expand the permission macros. The first permission macro that we shall expand will be read_file_perms. The "read_file_perms" permission macro is defined in the file: */usr/share/selinux/devel/include/support/obj_perm_sets.spt*

```
define(`read_file_perms',`{ open read_inherited_file_perms }')
```

The above m4 language directive and syntax could be familiar to sendmail administrators, if they use macros for generating the sendmail configuration file. In plain words, define is the keyword; the first parameter is the macro being defined, and the second parameter is the expansion.

From the above, we see that the macro "read_file_perms" refers to another macro "read_inherited_file_perms". Let us look at its definition. Fortunately, it is defined in the same file:

```
define(`read_inherited_file_perms',`{ getattr read ioctl lock }')
```

Therefore, our permission macro "read_file_perms" can be expanded to "{ open getattr read ioctl lock }". These are all standard permissions.

Replacing the permission macro "read_file_perms" in our example.te, the allow rule,

```
allow myapp_t myapp_log_t:file { read_file_perms append_file_perms };
```

can be re-written as:

```
allow myapp_t myapp_log_t:file { open getattr read ioctl lock append_file_
perms };
```

Expanding the permission macro "append_file_perms"